

為了應對完全隨機的 Gridworld (Random Mode)——其中玩家、終點、陷阱與牆壁的位置在每一次訓練都會重新洗牌，我們將 PyTorch 實作轉換，並比較了 Keras 與 PyTorch Lightning 兩種高階框架，同時整合了進階的訓練技巧來穩定模型。

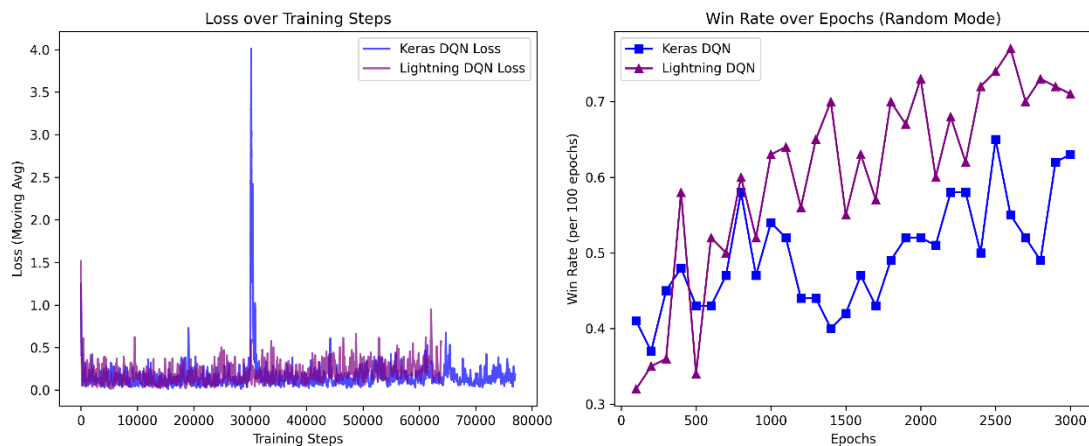
1. 框架實作比較：

- **Keras**：透過 `@tf.function` 進行圖編譯加速。在自定義 Training Loop 中，使用 `tf.GradientTape` 來精準控制梯度的計算，確保 Target Y 的計算不參與反向傳播。
- **PyTorch Lightning**：將繁瑣的 Boilerplate Code（如迴圈、梯度清零、設備轉換）封裝。透過實作 `LightningModule` 與自定義的 `IterableDataset` 來管理 RL 的資料流，程式碼結構更具模組化與可讀性。

2. 整合的 Training Tips (穩定訓練的關鍵)：

- **Gradient Clipping (梯度裁剪)**：在 Random Mode 中，極端隨機的狀態變化容易產生巨大的 TD Error，導致梯度爆炸。透過實作梯度裁剪（Keras 的 `clipnorm=1.0` 與 Lightning 的 `gradient_clip_val=1.0`），我們將梯度的範數限制在安全範圍內，確保網路權重不會被破壞。
- **Learning Rate Scheduling (學習率排程)**：Keras 使用 `ExponentialDecay`，Lightning 使用 `StepLR`。這讓 Agent 在訓練初期以較大參數（如 10^{-3} ）快速探索環境；中後期自動遞減學習率，幫助模型在最佳解附近微調（Fine-tuning），有效減少 Random Mode 帶來的 Loss 震盪。

根據 hw3-3_metrics_comparison.jpg，我們對比了兩種框架在極端隨機環境下的收斂表現：



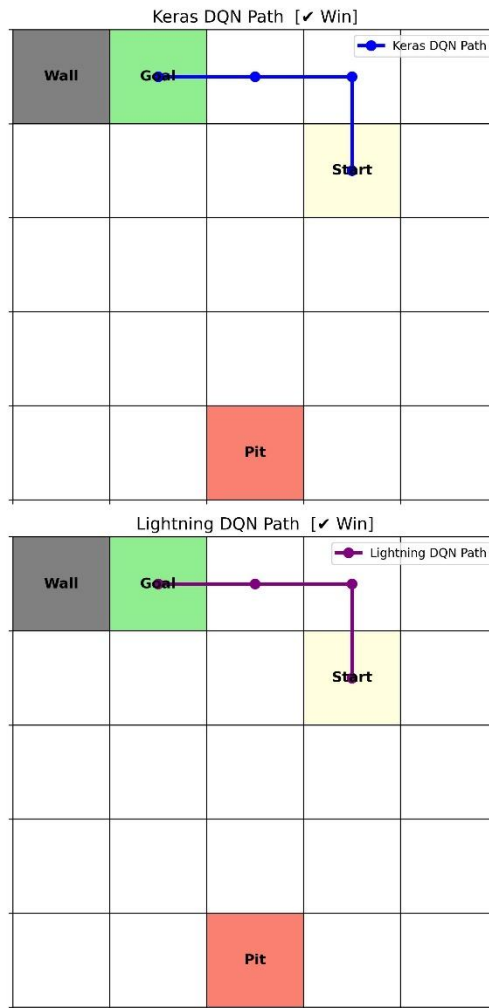
1. Win Rate over Epochs (勝率表現)：

- 在 **Random Mode** 中，由於所有物件位置都不固定，勝率無法像前兩個階段一樣迅速飆升至 **100%**。但我們可以看到兩種框架都展現了穩健的學習能力，勝率呈現穩定向上的趨勢。
- 經過 **3000** 個 **epochs** 的訓練，**PyTorch Lightning**（紫色三角線）的勝率來到了接近 **80% (0.75~0.8)** 的高水準；**Keras**（藍色方塊線）也達到了近 **70% (0.65~0.7)**。這顯示模型已經從隨機遊走中，真正學會了「尋找目標與避開危險」的通用法則。

2. Loss over Training Steps (損失函數與裁剪驗證)：

- **Loss** 曲線整體呈現密集的微小震盪，這是 **Random Mode** 環境不確定性的正常現象。
- **關鍵觀察**：在 **Keras** 的 **Loss** 曲線中，大約在 **30,000 steps** 時出現了一個極其突兀的巨大波峰（**Loss** 飆高至 **4.0**）。如果沒有實作 **Gradient Clipping**，這種異常的誤差突波極有可能直接摧毀神經網路的權重，導致訓練發散（**Catastrophic Forgetting**）。然而，模型在經歷波峰後迅速回穩，完美驗證了梯度裁剪機制在複雜環境中的必要性與成效。

觀察 hw3-3_paths_comparison.jpg 的實際測試結果，我們檢驗了模型是否具備「舉一反三」的泛化能力：



- **隨機生成的測試環境**：本次測試中，Goal 隨機生成於 (0,1)，Wall 位於左上角 (0,0)，Pit 位於下方 (4,2)，而 Start 則生成於右方 (1,3)。
- **策略高度一致且最佳化**：即使面對一個訓練過程中可能從未見過的全新地圖配置，Keras DQN 與 Lightning DQN 都成功達成了 [✓ Win]。
- **路徑解讀**：兩個模型都做出了極具邏輯的決策——從 Start (1,3) 出發，先向上移動至安全的頂部邊緣 (0,3)，接著毫無猶豫地向左直線前進，經由 (0,2) 直接抵達 Goal (0,1)。它們完全沒有往下走靠近 Pit 的風險區域。這證明了在 Keras 與 Lightning 框架下，配合良好的學習率排程，DQN 確實學會了「看懂地圖」，具備了真正的空間感知與規劃能力。